

Memory address

- * When we create a variable, memory address is assigned to the variable.
- * Memory address is the location of where the variable stored in the computer.
- * When we assign a value to variable, it stored on the memory address.
- * To access, we can use 'reference operator' (&)

Ex.

```
int myAge = 43;
```

```
printf("%p", &myAge); // Outputs 0x7ffe5367e044
```

The memory address.

(This can be change in the different computers)

Note: Memory addresses in the form of hexadecimal.

Note: "%p" is the format specifier for the 'pointers'.

- ⊛ Knowing the memory address useful to reduce the code and improve the performance.

Pointers

- * A 'Pointer' is a variable that stores the memory address of another variable.
- * Pointer variable points to a data type of the same type, and it created with the operator '*'

Ex.

```
int myAge = 43;
```

```
int* ptr = &myAge; // Pointer variable 'ptr' stores the memory address of 'myAge' as it's value.
```

```
printf("%p\n", &myAge); // Output the memory address of 'myAge'
```

```
printf("%p\n", ptr); // Output the memory address of 'myAge' with the pointer
```

Note: The type of the pointer has to match the type of the variable. (int in above example)

Use the '&' operator to store the memory address of the 'myAge' variable and assign ~~in~~ it to the pointer.

Note: There are two ways to declare pointer variable in C.

① `int* myNum;`

② `int *myNum;`

Be Careful; when using pointers. Because, it is possible to damage data stored in other memory address.

Functions

- * A function is a block of code which only runs when it is called.
- * We can pass data into a function, known as parameters.
- * Functions are used to do a specific operation and they are ~~important~~ important to reuse the same code.
- * There are two main types of functions.
 - (i) Predefined functions (Build-in functions)
 - (ii) User-defined functions
- * 'Predefined functions' are available in standard C library and any time / any where we can use them to do our operations.
(E.x. `main()`, `printf()`, `scanf()`)
- * 'User-defined functions' are defined by the programmer to do a specific task.
- ④ To create a function, we should have to follow the below structure.

```
void myFunction() {  
    // Code to be executed  
}
```

Diagram labels:

- return type of the function (points to `void`)
- name of the function (points to `myFunction`)
- parentheses (points to `()`)
- // Code to be executed (points to the code block)

Global Scope

Note: return type can be changed. This "void" means that this function does not have a return value.

- ★ Declared functions are not executed immediately, and they will be executed when they are called.
- ★ To call a function, write the function's name followed by two parentheses (()) and a semicolon (;)

E.x.

```
① // Create a function
void myfunction () {
    printf("This is my first function");
}
int main () {
    myfunction(); // Call the function

    return 0;
}
// Outputs "This is my first function"
```

Function parameters and arguments

- ★ Information can be passed to functions as a parameter.
- ★ Parameters act as variables inside the function.
- ★ If you want to add many parameters, separate them with commas.
- ★ Structure as below;

```
return type    function name (parameter 1, parameter 2, ...) {
    // Code to be executed
}
```

E.x. ①

```
void myfunction (char name[]) {
    printf("Hello %s\n", name);
}
int main () {
    myfunction("Liam"); // Output → Hello Liam
    return 0;
}
```

Note: When a parameter is passed to the function, it is called an argument. So, in above example "name" is the 'parameter' and "Liam" is an argument.

* Inside the function, you can add as many parameters as want;

E.x. ① void myfunction (char name[], int age) {
printf("Hello %.s. You are %d years old.\n",
name, age);

}

int main () {
myfunction("Liam", 3);

return 0;

}

// Outputs → Hello Liam. You are 3 years old.

Variable Scope

* In C language, variables are only accessible inside the region they are created. This called 'scope'.

Local scope

* A variable created inside a function belongs to the "local scope" of that function, and can only be used inside that function.

E.x. ① void myfunction () {
// Local variable that belongs to myfunction()

int x = 5; ←

printf("%d", x);

}

int main () {
myfunction ();

return 0;

}

★ We can only use this ~~function~~ variable inside this function.

Global Scope

- * A variable created outside of a function, is called a "global variable" and belongs to the global scope.

Ex: ① // Global variable x

```
int x = 5; ←
```

```
void myFunction() {  
    // We can use x here  
    printf("%d", x);  
}
```

```
int main() {  
    myFunction();
```

```
    // We can also use x here  
    printf("%d", x);
```

```
    return 0;
```

```
}
```

② We can use this variable anywhere of this program.

Math Functions

- * There are lot of math functions available in C, that allows to perform mathematical tasks on numbers.
- * To use them, must include "math.h" header file in program.

Ex:

```
#include <stdio.h>
```

```
#include <math.h>
```

```
int main() {
```

```
    // Code goes here
```

```
    return 0;
```

```
}
```

- ③ Below the list of most important Math functions,

$\text{cbrt}(x) \Rightarrow$ Returns the cube root of x

$\text{ceil}(x) \Rightarrow$ Returns the value of x rounded up to its nearest integer

$\text{cos}(x) \Rightarrow$ Returns the cosine of x (x is in radians)

$\text{fabs}(x) \Rightarrow$ Returns the absolute value of x

$\text{floor}(x) \Rightarrow$ Returns the value of x rounded down to its nearest integer.

$\text{fmax}(x, y) \Rightarrow$ Returns the highest value of a floating x and y

$\text{fmin}(x, y) \Rightarrow$ Returns the lowest value of a floating x and y

$\text{fmod}(x, y) \Rightarrow$ Returns the floating point remainder of x/y

$\text{log}(x) \Rightarrow$ Returns the natural logarithm of x

$\text{pow}(x, y) \Rightarrow$ Returns the value of x to the power of y

$\text{remainder}(x, y) \Rightarrow$ Return the remainder of x/y rounded to the nearest integer.

$\text{round}(x) \Rightarrow$ Returns x rounded to the nearest integer.

$\text{sin}(x) \Rightarrow$ Returns the sine of x (x is in radians)

$\text{tan}(x) \Rightarrow$ Returns the tan of x (x is in radians)

$\text{sqrt}(x) \Rightarrow$ Returns the square root of x

$\text{trunc}(x) \Rightarrow$ Returns the integer part of x

Return Statement

* Return statement used within functions to control the execution flow and send a value back to the calling program or current function.

① Return with an Expression;

Syntax. return Expression;

⊛ In this form, the expression represents the value that the C function returns to the calling code. This value could be a 'constant', 'variable' or 'the result of a computation'.

② Return with a value;

Syntax. return;

⊛ When a function has no return value or is used to perform a task without providing any output, the return statement can be used without an expression. This form simply indicates the end of the function's execution.

③ No return statement;

Syntax. void myfunction()

⊛ If a function does not require a return statement, it can be defined with a return type of void, indicating that the function does not return any value.